Check out our **Cloud Native Services** and book a call with one of our experts today!

CLOUD NATIVE,BUILD,DESIGN

How to Create Cross-Account User Roles for AWS with Terraform

Charlenne Mach

April 7, 2020
7 minutes Read

Most companies these days use multiple cloud accounts to separate resources, customers, or even internal departments. With multiple AWS accounts, it's practical to rely on a so-called bastion account for Identity and Access Management (IAM) users. It serves as one central place for users, S3 buckets, and other shared resources.

If you are not using AWS Organizations, you can follow the best practices guide for multi-account setups here.

While you could also just replicate your users across those other accounts, the simplest and cleanest way to access any resources there is to use AWS roles. Roles enable users and AWS services to access other AWS accounts without having to create a user in those accounts first. This post shows how to set up access to resources in another account via Terraform.

Gaining Trust

The way roles work is by using a web service called AWS Security Token Service (STS) to request temporary credentials for IAM, which are then used to identify you as that role. A user can request access to a role, which will grant that user that role's temporary privileges.

For that to be secure, there needs to be a trust established between the account or user and the role. It is possible to set up a role without restrictions that anyone can use, but that's very insecure and not recommended.

Trust works by defining a policy to make that role assumable by only certain users, as well as a policy to allow only certain users to assume that role, taking care of permissions in both accounts. This might seem like doing the same thing twice, but you're actually establishing the trust from both sides by setting those two policies.

Setting Up the Role

In this example, we're setting up a user in an AWS account we'll call 'utils':

```
resource "aws_iam_user" "random" {
  name = "random_user"
  tags = {
    name = "random"
  }
}
```

We're giving it the right to assume a specific role in another account.

```
resource "aws_iam_policy" "prod_s3" {
  name        = "prod_s3"
  description = "Allow assuming prod_s3 role"
  policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Effect   = "Allow",
        Action   = "sts:AssumeRole",
        Resource = "arn:aws:iam::${data.aws_caller_identity.prod.accou
      }
    ]
  })
}

resource "aws_iam_user_policy_attachment" "prod_s3" {
  user       = aws_iam_user.random_name
  policy_arn = aws_iam_policy.prod_s3.arn
}
```

Since we're using the same Terraform for two AWS accounts, we're defining a second provider, which is then used to make sure the next resources get created in the second account instead of the first.

```
provider "aws" {
  version = "~> 2.49"
  profile = "utils"
  region  = var.region_utils
}
provider "aws" {
  version = "~> 2.49"
  profile = "prod"
  region  = var.region_prod
  alias   = "prod"
}
```

In the second account (let's call it 'prod'), we're creating a role with a policy to allow that role to be assumed from the utils account.

```
resource "aws_iam_role" "prod_list_s3" {
  name = "s3-list-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17",
    Statement = [
      {
        Effect   = "Allow",
        Action   = "sts:AssumeRole",
        Principal = { "AWS" : "arn:aws:iam::${data.aws_caller_identity
      }
    ]
  })
  provider = aws.prod
}
```

We are also adding a policy to grant the newly created role some permissions in the prod account. In this case, we're only letting it list a few S3 buckets.

We create a JSON file for the S3 permissions, called "role_permissions_policy.json". Those could be done inline like the other policies, but having them separate makes the Terraform files easier to read —especially with longer statements.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "s3:ListAllMyBuckets",
            "Resource": "*"
        }
    ]
}
```

Back in the Terraform files we attach that policy (by referring to the JSON file) to the role we created before.

```
resource "aws_iam_policy" "s3_list_all" {
  name        = "s3_list_all"
  description = "Allows listing all s3 buckets"
  policy      = file("role_permissions_policy.json")

  provider = aws.prod
}

resource "aws_iam_policy_attachment" "s3_list_all" {
  name       = "list s3 buckets policy no role"
  roles      = ["${aws_iam_role.prod_list_s3.name}"]
  policy_arn = aws_iam_policy.s3_list_all.arn
  provider   = aws.prod
}
```

The complete files can also be found in this repository.

Now apply those Terraform files by running `terraform init` and then `terraform apply`.

Assuming the Role

If you want to use the newly created user, add a password to it and login as that user into the utils account.

Try out the role to access the S3 buckets in prod by following the steps in the documentation.

Alternatively use the AWS CLI

1. Get the role ARN.

```
aws iam list-roles --query 'Roles[?RoleName == `s3-list-role`].[RoleNa
```

2. Request STS from AWS using the role ARN and a session name of your choosing.

```
aws sts assume-role --role-arn "<role_arn>" --role-session-name "<some
```

3. User gets temporary credentials, export these as environment variables.

```
export AWS_ACCESS_KEY_ID=<exampleAccessKeyID
export AWS_SECRET_ACCESS_KEY=<exampleSecretKey
export AWS_SESSION_TOKEN=<exampleSessionToken
```

4. Access S3 using the temp credentials.

```
aws s3api list-buckets
```

To-do! We can now login into our utils account, assume the role, and look at the prod S3 buckets.

Of course this is a fairly simple example, but roles are also immensely useful for granting temporary access or allowing users to switch between different accounts and permission levels quickly. Admins can check user permissions without logging in and out, developers can access different accounts without changing users, and pipelines can function across AWS accounts without multiple sets of access keys.

The Cloud Native Attitude, 2nd Edition
By Anne Currie and Charles Humble

A concise distillation of what we gleaned from our conversations with users, vendors, hosting providers, journalists, researchers, and our own clients.

Download your **FREE** copy

Leaders: How Well Do You Balance Your Head and Heart?

← Previous article

Next article →

How to Manage Distributed Teams (Even When You Have No Choic...

Leave your Comment

Your name

Your email address

Your website (optional)

Your comment

Container Solutions needs the contact information you provide to us to contact you about our products and services. You may unsubscribe from these communications at any time. For information on how to unsubscribe, as well as our privacy practices and commitment to protecting your privacy, please review our Privacy Policy.

Post your comment

© 2024 Container Solutions
All rights reserved

Sign up for our CS newsletter
Take a deep dive into tech, strategy, culture, and more.
Subscribe now